



Comandos de manejo de archivos y filesystem.

- **cd:** Cambia el directorio actual.
- **cp:** Copia archivos o directorios.
- **cut:** Corta columnas o campos de texto.
- **df:** Muestra el espacio disponible en discos.
- **du:** Muestra el espacio usado por archivos/directorios.
- **fdisk:** Manipula particiones de disco.
- **gzip:** Comprime archivos en formato .gz.
- **ln:** Crea enlaces duros o simbólicos.
- **losetup:** Asocia archivos a dispositivos de bucle.
- **ls:** Lista archivos y directorios.
- **mkdir:** Crea directorios.
- **mkfs:** Crea un sistema de archivos (formatea).
- **mount:** Monta sistemas de archivos.
- **mv:** Mueve o renombra archivos o directorios.
- **pwd:** Muestra el directorio actual.
- **read:** Lee una línea desde entrada estándar.
- **rm:** Borra archivos
- **rmdir:** Borra directorios vacíos.
- **stat:** Muestra información detallada de un archivo.
- **tar:** Empaqueta y/o comprime archivos.
- **touch:** Crea un archivo vacío o actualiza su fecha.
- **tree:** Muestra la estructura de directorios en forma de árbol.
- **umount:** Desmonta sistema de archivos
- **wc:** Cuenta líneas, palabras, caracteres.
- **zip:** Comprime archivos en formato ZIP.
- **zgrep:** Hace grep dentro de archivos comprimidos con gzip.
- **basename:** devuelve el nombre del archivo quitando la ruta o también la extensión si se pasa como segundo argumento.



Comandos para visualizar archivos.

- **cat:** Muestra (imprime) el contenido de un archivo.
- **echo:** Imprime texto.
- **head:** Muestra las primeras líneas de un archivo.
- **less:** Muestra el contenido paginado hacia adelante/atrás.
- **more:** Muestra el contenido paginado hacia adelante.

- **tail:** Muestra las últimas líneas de un archivo.
- **type:** Muestra si una palabra es un comando interno, alias, función, etc.

Comandos de búsqueda.

- **find:** Busca archivos por nombre, tamaño, permisos, etc.
- **grep:** Busca texto dentro de archivos.
- **locate:** Busca archivos rápidamente, usando base de datos.
- **pgrep:** Busca procesos por nombre.
- **zgrep:** Igual que grep pero en archivos comprimidos.

Comandos sobre procesos y rendimiento

- **atop:** Monitor avanzado de rendimiento.
- **htop:** Versión mejorada de top.
- **kill:** Mata un proceso por PID.
- **killall:** Mata procesos por nombre.
- **nice:** Ejecuta un comando con prioridad modificada.
- **pkill:** Mata procesos que coincidan con un patrón.
- **ps:** Lista procesos en ejecución.
- **pstree:** Muestra procesos en forma de árbol.
- **renice:** Cambia la prioridad.
- **top:** Monitor del sistema en tiempo real.
- **xkill:** Cierra una ventana haciendo clic encima.

Comandos de usuarios y permisos

- **chgrp:** Cambia el grupo de un archivo.
- **chmod:** Cambia permisos de un archivo.
- **chown:** Cambia el dueño de un archivo.
- **groupadd:** Crea un grupo nuevo.
- **passwd:** Cambia la contraseña de un usuario.

- **su**: Cambia de usuario.
- **useradd**: Crea un usuario nuevo.

Comandos de sistema e información.

- **man**: Abre el manual de un comando.
- **uname**: Muestra información del sistema (arquitectura, Kernel).
- **who**: Muestra quien esta logueado en el sistema.
- **write**: Envía mensajes a otros usuarios del sistema.

Edición de texto.

- **nano**: Editor de texto simple en terminal.
- **vim**: Editor de texto avanzado y potente.

Comandos de Red

- **ip**: Configura y muestra información de red.
- **netstat**: Muestra conexiones de red, puertos y sockets.

Redirecciones / Utilidades.

- **>**: Redirige la salida creando y sobrescribiendo.
- **>>**: Redirige la salida agregando al final, sin borrar.
- **<**: Redirige la entrada, toma la entrada de un archivo.
- **tee**: Muestra la salida y la guarda en un archivo.
- **|**: Envía la salida de un comando como entrada del siguiente.
- **2>**: Redirige errores estándar (stderr).

Sistema de Ficheros Linux (FHS)

- **/boot**: Contiene los cargadores de arranque y el kernel. Sin esto el sistema no inicia.
- **/bin**: Programas esenciales del sistema, accesibles para todos los usuarios (ej: ls, cp, mv).
- **/sbin**: Programas esenciales solo para administrador (ej: fsck)
- **/usr**: Uno de los directorios más grandes: contiene programas, librerías y documentación de usuario.
 - /usr/bin** → Programas no esenciales.
 - /usr/sbin** → Herramientas administrativas no esenciales.
 - /usr/lib** → Librerías.
 - /usr/share** → Documentación, íconos, manuales.
- **/lib**: Librerías esenciales para binarios en /bin y /sbin.
 - /lib64**: Librerías para sistemas de 64 bits (si se posee).
- **/dev**: Dispositivos representados como archivos: discos, terminales, USB, etc.
- **/etc**: Archivos de configuración del sistema.
 - /etc/passwd** → usuarios.
 - /etc/fstab** → puntos de montaje.
 - /etc/hostname** → nombre del sistema.
- **/home**: Directorios personales de cada usuario.
- **/root**: Directorio personal del usuario root.
- **/tmp**: Archivos temporales, cualquier usuario puede escribir aquí.
- **/var**: Contiene datos variables que cambian durante el funcionamiento del sistema:
 - /var/log** → logs del sistema.
 - /var/lib** → bases de datos de servicios.
- **/run**: Contiene archivos temporales de estado del sistema desde el arranque (PID, sockets).
- **/srv**: Datos “servidos” por el sistema, por ejemplo: contenido de un servidor web.
- **/media**: Puntos de montaje automáticos: pendrives, CD, discos externos.
- **/sys**: Sistema de archivos virtual para interactuar con dispositivos y el kernel.

Estructuras de control

Selección de alternativas:

Decisión:

```
if [ condition ]
then
    bloque
elif [ condition ]
then
    bloque
else
    bloque
fi
```

Selección:

```
case $variable in
"valor 1")
    bloque
;;
"valor 2")
    bloque
;;
*)
    bloque
;;
esac
```

Menú de opciones:

Ejemplo:

```
select accion in Nuevo Salir
do
    case $accion in
        "Nuevo")
            echo "Seleccionado: Nuevo"
            ;;
        "Salir")
            exit 0
            ;;
    esac
done
```

Iteración:

C-style:

```
for ((i=0; i < 10; i++))
do
    bloque
done
```

Con lista de valores (foreach):

```
for i in valor1 valor2 valor3 valorN
do
    bloque
done
```

While

```
while [ condicion ] # Mientras se cumpla la condición
do
    bloque
done
```

Until

```
until [ condition ] # Mientras NO se cumpla la condición
do
    bloque
done
```

Funciones

```
# Recibe 2 argumentos y devuelve:
# 1 si el primero es el mayor
# 0 en caso contrario

mayor() {
    echo "Se van a comparar los valores: $*"
    if [ $1 -gt $2 ]; then
        echo "$1 es el mayor"
        return 1
    fi
    echo "$2 es el mayor"
    return 0
}

mayor 5 6      # Invocación
echo $?        # Imprime el exit status de la función
```

Evaluación de condiciones lógicas

Operador	Con strings	Con números
Igualdad	"\$nombre" = "Maria"	\$edad -eq 20
Desigualdad	"\$nombre" != "Maria"	\$edad -ne 20
Mayor	A > Z	5 -gt 20
Mayor o igual	A >= Z	5 -ge 20
Menor	A < Z	5 -lt 20
Menor o igual	A <= Z	5 -le 20

Comando test expresión.

El comando test permite evaluar:

- Expresiones de archivos.
- Expresiones de cadenas de texto.
- Expresiones numéricas.

El resultado será: 0 (true) si la expresión es verdadera. 1 (false) si es falsa.

Evaluación de archivos

Sirven para comprobar la existencia, tipo o permisos de un archivo/directorio.

Comprobación de existencia y tipo:

-e archivo	El archivo existe (exists)
-f archivo	Existe y es un archivo regular (file)
-d archivo	Es un directorio (directory)
-l archivo	Es un enlace simbólico (link)
-s archivo	Existe y tiene tamaño mayor a cero (size)

Permisos:

-r archivo	Tiene permiso de lectura (read)
-w archivo	Tiene permiso de escritura (write)
-x archivo	Tiene permiso de ejecución (execute)

Comparación entre archivos:

archivo1 -nt archivo2	archivo1 es más nuevo (newer than)
archivo1 -ot archivo2	archivo1 es más viejo (older than)
archivo1 -ef archivo2	Son el mismo archivo (equal file)

Evaluación de cadenas de caracteres

Longitud:

-z cadena	Longitud cero (cadena vacía) (zero)
-n cadena	Longitud mayor a cero (non-cero)

Comparación de cadenas:

cadena1 = cadena2	Son iguales
cadena1 != cadena2	Son distintas

Evaluaciones numéricas

Se usan operadores específicos para enteros.

-eq	Igual que (equal)
-ne	Distinto (not equal)
-gt	Mayor que (greater than)
-lt	Menor que (less than)

-ge	Mayor o igual que (greater or equal)
-le	Menor o igual que (less or equal)

Variables.

Declaración de variables

En Bash, se declaran simplemente asignando un valor sin espacios:

```
nombre="Carlos"
edad=25
```

Referencia a variables

Para usar una variable, se antepone el símbolo \$:

```
echo "Nombre: $nombre"
echo "Edad: $edad"
```

También se pueden usar llaves para evitar ambigüedades:

```
echo "Hola ${nombre}"
```

En bash, las variables no tienen tipo de dato fijo. Todo se maneja como cadenas de texto, (bash permite interpretarlas como números en contextos aritméticos).

Tipos según uso:

Variables de texto (por defecto todo es texto)

- Variables numéricas
- Variables de entorno
- Variables especiales (ej.: \$?, \$0, \$1, \$#, etc.)
- Arreglos (arrays indexados)

Bash No es un lenguaje fuertemente tipado. No se necesita declarar el tipo de una variable. Se puede asignar números, textos, rutas, etc.

Arreglos

• Creación:

```
arreglo_a=()          # Se crea vacío
arreglo_b=(1 2 3 5 8 13 21) # Inicializado
```

• Asignación de un valor en una posición concreta:

```
arreglo_b[2]=spam
```

• Acceso a un valor del arreglo (en este caso las llaves no son opcionales):

```
echo ${arreglo_b[2]}
copia=${arreglo_b[2]}
```

• Acceso a todos los valores del arreglo:

```
echo ${arreglo[@]}      # o bien ${arreglo[*]}
```

• Tamaño del arreglo:

```
${#arreglo[@]}          # otra forma ${#arreglo[*]}
```

• Borrado de un elemento (reduce el tamaño del arreglo, pero no elimina la posición, solamente la deja vacía):

unset arreglo[2]

- Los índices en los arreglos comienzan en 0

Funciones.

Las funciones permiten modularizar el comportamiento de los scripts.

- Se pueden declarar de 2 formas:

- **function nombre { bloque }**
- **nombre() { bloque }**

- Con la sentencia **return** se retorna un valor entre 0 y 255
- El valor de retorno se puede evaluar mediante la variable **\$?**
- Reciben argumentos en las variables **\$1**, **\$2**, etc.

Los parámetros de una función no se declaran, sino que se reciben igual que los parámetros de un script.

\$1 -> primer argumento.

\$2 -> segundo argumento.

\$@ -> todos los argumentos.

\$# -> cantidad de argumentos.

Ejemplo:

```
saludar() {  
    echo "Hola $1, tu edad es $2"  
}  
saludar "Juan" 25
```

Salida: **Hola** Juan, tu edad es 25

Comando expr.

El comando **expr** en **bash** permite evaluar expresiones aritméticas, relacionales, lógicas y de manipulación de cadenas.

- Operaciones aritméticas:

Permite realizar cálculos numéricos.

+ Suma	expr 4 + 5 → 9
- Resta	expr 10 - 3 → 7
* Multiplicación	expr 4 * 5 → 20
/ División entera	expr 7 / 2 → 3
% Módulo	expr 10 % 3 → 1

- Operaciones relacionales:

Usadas para comparar valores (retornan 1 = verdadero, 0 = falso).

= Igual	expr 5 = 5 → 1
\!= Distinto	expr 5 \!= 3 → 1
> Mayor que	expr 7 > 2 → 1
< Menor que	expr 2 < 1 → 0

>= Mayor o igual

expr 3 >= 3 → 1

<= Menor o igual

expr 1 <= 5 → 1

- Operaciones con cadenas:

Permite trabajar con texto.

▪ Comparación de cadenas

expr "hola" = "hola"

▪ Longitud de una cadena

expr length "texto" → 5

▪ Substring (extracto)

expr substr "computadora" 2 4 → "ompu" (Posición inicial: 1)

▪ Índice de un carácter dentro de una cadena

expr index "banana" a



Parametrización

Cuando se ejecuta un script como:

```
./mi_script.sh arg1 arg2 arg3
```

Dentro del script se puede acceder a cada parámetro posicional usando:

\$0 → el nombre del script

\$1 → primer argumento (arg1)

\$2 → segundo argumento (arg2)

\$3 → tercer argumento (arg3)

... y así sucesivamente.

\$# - Numero de parámetros: Devuelve cuantos argumentos fueron pasados al script.

```
echo "Cantidad de parámetros: $#"
```

\$* - Todos los parámetros como una sola cadena: Contiene todos los argumentos, expandidos como una sola palabra.

```
echo "Parámetros: $*"
```

\$? - Código de salida del ultimo comando: Muestra si el ultimo comando se ejecutó correctamente

0 -> éxito

cualquier otro número -> hubo error

\$HOME - Directorio home del usuario: Esta variable de entorno contiene la ruta del directorio personal del usuario actual.